

Technická univerzita v Košiciach
Fakulta elektrotechniky a informatiky
počítačov a informatiky

Zvýšenie bezpečnosti vybraných šifrovacích algoritmov v prístupových systémoch

Diplomová práca

ACCESS-SECURITY

Používateľská príručka

Vedúci diplomovej práce:

doc. Ing. Eva Chovancová, PhD.

Diplomant:

Oleksandr Mykhailyshyn

Konzultant diplomovej práce:

Košice 2026

Contents

1	Funkcia programu	1
2	Súpis obsahu dodávky	1
3	Inštalácia programu	2
3.1	Požiadavky na technické prostriedky	2
3.2	Požiadavky na programové prostriedky	2
3.3	Vlastná inštalácia	2
3.4	Popis štruktúry programu	3
4	Použitie programu	4
4.1	Popis dialógu s používateľom	4
4.2	Popis funkcií a podfunkcií programu	4
5	Popis vstupných, výstupných a pracovných súborov	5
6	Obmedzenia programu	6
7	Chybové hlásenia	6
8	Príklad použitia	7

1 Funkcia programu

Access-Security je prototyp systému kontroly fyzického prístupu na báze RFID/NFC. Hardvérová čítačka (ESP32 + MFRC522) prečíta UID karty a odošle autentifikovanú HTTP požiadavku serveru, ktorý na základe roly karty a oprávnení dverí rozhodne o povolení alebo zamietnutí prístupu. Komunikácia je chránená HMAC podpisom, AEAD šifrovaním (ChaCha20-Poly1305), anti-replay mechanizmom a detektorom protokolových anomálií; všetky udalosti sú zaznamenané v tamper-evident auditnom logu.

Program je určený na laboratórne nasadenie a experimentálnu validáciu kryptografických mechanizmov; produkčné nasadenie vyžaduje dodatočné kroky uvedené v kapitole *Obmedzenia programu*. Detailný technický popis je v systémovej príručke priloženej k práci.

Autor: Oleksandr Mykhailyshyn, FEI TUKE, 2026.

Licencia: zdrojový kód aj všetky externé závislosti sú dostupné pod permissívnymi open-source licenciami (MIT, BSD, ISC, Apache 2.0).

2 Súpis obsahu dodávky

Dodávka pozostáva z jedného elektronického média (USB / CD), ktoré obsahuje:

- zdrojový kód projektu (vetva **analytical** Git repozitára, vrátane produkčného serverového kódu, firmvéru ESP32, experimentálneho harnessu, analytického pipeline a konfiguračných súborov);
- **Dockerfile** pre kontajnerizované nasadenie;
- dokumentáciu: text diplomovej práce, systémovú príručku a túto používateľskú príručku (vo formáte PDF aj v zdrojovom L^AT_EX formáte);

- súbor `README.md` v koreňovom adresári so stručným prehľadom projektu.

Master kľúč (`secrets/master_key.hex`) nie je súčasťou dodávky — musí byť vygenerovaný pri prvom nasadení (popísané v podkapitole 3.3).

3 Inštalácia programu

3.1 Požiadavky na technické prostriedky

Server: x86-64/ARM64, min. 2 jadrá, 2 GB RAM, 1 GB diskového priestoru pri preklade (150 MB pre prevádzku). Hardvér: ESP32 DevKit, modul MFRC522, zelená LED, piezomenič a USB kábel pre nahranie firmvéru.

3.2 Požiadavky na programové prostriedky

Linux Ubuntu 22.04+, GCC ≥ 12 , CMake ≥ 3.20 , `libsqlite3-dev`, Python ≥ 3.10 (pre analytický pipeline). Pre firmvér Arduino IDE 2.x s podporou ESP32 a knižnicou MFRC522. Voliteľne Docker Engine 20.10+ pre kontajnerizované nasadenie.

3.3 Vlastná inštalácia

Server. V koreňovom adresári projektu:

```
mkdir build && cd build
cmake ..
cmake --build . -j$(nproc)
```

Pri prvom zostavení CMake FetchContent automaticky stiahne externé závislosti (`libsodium`, `cpp-httpplib`, `yaml-cpp`, `nlohmann/json`, `Google Test`). Verifikácia: `ctest -output-on-failure`.

Master kľúč. Pred prvým spustením vygenerovať 32-bajtový kľúč:

```
mkdir -p secrets  
openssl rand -hex 32 > secrets/master_key.hex
```

Firmvér ESP32. V Arduino IDE upraviť `hw_layer/hw_config.h` (Wi-Fi SSID/heslo, URL servera, HMAC secret, identita čítačky), zvoliť board *ESP32 Dev Module* a nahráť skech `hw_layer.ino` cez USB.

Docker (alternatíva). `docker build -t access-security .` a následne `docker run -p 8080:8080 -v ./data:/app/data -v ./secrets:/app/secrets access-security`.

3.4 Popis štruktúry programu

Po spustení servera (binárny súbor `access_admin` s argumentom YAML konfigurácie) sa otvoria dva HTTP porty: 8080 (frontend a webové rozhranie) a 8081 (DecisionService, len loopback). Webové administrátorské rozhranie je prístupné na adrese `localhost:8080/static/index.html`.

Typický tok spracovania požiadavky:

1. Používateľ priloží kartu k čítačke.
2. Firmvér prečíta UID, vypočíta HMAC podpis a odošle HTTP požiadavku na port 8080.
3. Frontend služba overí HMAC, vytvorí AEAD frame a odošle ho na DecisionService (port 8081).
4. DecisionService dešifruje frame, overí anti-replay, spustí anomálny detektor, vyhladá kartu v DB a vráti rozhodnutie.

5. Frontend odošle JSON odpoveď firmvéru, ktorý signalizuje výsledok (zelená LED + krátke pípnutie pri povolení; dvojité pípnutie pri zamietnutí).

Detailný popis architektúry, modulov a tokov je v systémovej príručke.

4 Použitie programu

Po inštalácii a spustení servera má používateľ dva spôsoby interakcie: koncový používateľ (držiteľ karty) komunikuje so systémom výhradne cez priloženie karty k čítačke; administrátor systém spravuje cez webové rozhranie.

4.1 Popis dialógu s používateľom

Koncový používateľ (držiteľ karty). Po priložení karty čítačka signalizuje výsledok:

- *Povolený prístup:* jedno krátke pípnutie (2 kHz) so svetlom zelenej LED.
- *Zamietnutý prístup:* dvojité pípnutie nižšieho tónu (500 Hz).

Administrátor. Otvorí v prehliadači adresu `localhost:8080/static/index.html`, zadá administrátorský token (predvolene `admin`) a prepína medzi sekciami pomocou navigačného menu.

4.2 Popis funkcií a podfunkcií programu

Webové rozhranie ponúka šesť hlavných sekcií:

- *Readers* — registrácia čítačiek, priradenie verzie kľúča, zrušenie karantény.
- *Door roles* — definícia rolí pre dvere (RBAC pravidlá).
- *Cards* — pridávanie/odoberanie kariet a ich rolí (UID sa ukladá ako HMAC odtlačok).

- *Audit* — prehliadanie auditného logu a kryptografická verifikácia integrity reťaze (auto-refresh každú sekundu).
- *Runtime logs* — prevádzkové udalosti v reálnom čase (polling 500 ms).
- *DB Export/Import* — záloha a obnova databázy (s kontrolou integrity `PRAGMA integrity_check`).

Každá operácia sa vykonáva cez REST endpoint chránený hlavičkou `X-Admin-Token`; výsledok sa zobrazí v príslušnej tabuľke alebo stavovom paneli.

5 Popis vstupných, výstupných a pracovných súborov

Vstupné:

- `config/access_security.yaml` — hlavná konfigurácia (port, anti-replay, decision mode).
- `secrets/master_key.hex` — 32-bajtový master kľúč v hex formáte (64 znakov).
- `hw_layer/hw_config.h` — konfigurácia firmvéru (kompiluje sa do binárneho súboru).

Výstupné a pracovné:

- `data/access.db` — SQLite databáza (čítačky, karty, RBAC, audit log); vytvorí sa automaticky pri prvom spustení.
- HTTP odpovede vo formáte JSON (`{"allow": bool, "reason": string}`).

Detailný popis formátov a štruktúr je v systémovej príručke.

6 Obmedzenia programu

Aktuálna verzia je laboratórny prototyp. Pred produkčným nasadením je potrebné brať do úvahy:

- *Bez TLS*: komunikácia medzi čítačkou a serverom prebieha cez nešifrovaný HTTP. HMAC zabezpečuje integritu a autentifikáciu, ale nie dôvernosť.
- *Master kľúč na disku*: `master_key.hex` je obyčajný súbor; pre produkciu sa odporúča HSM alebo vault.
- *Karta = identifikátor, nie kryptografický dôkaz*: klonovanie karty nie je riešené na úrovni protokolu.
- *Karanténa bez auto-zrušenia*: zakaranténovaná čítačka zostáva blokováná až do manuálneho zásahu administrátora.
- *Sériový debug výstup ESP32*: firmvér vypisuje citlivé údaje (UID, HMAC podpis) na sériovú konzolu — v produkcii odporúčané vypnúť.
- *Bez offline režimu*: pri výpadku servera čítačka nemá lokálnu cache oprávnení (fail-closed).

Detailné zhodnotenie obmedzení a smery ďalšieho vývoja sú v systémovej príručke a v kapitole 4 hlavnej diplomovej práce.

7 Chybové hlásenia

Server pri zamietnutí požiadavky vracia v JSON odpovedi pole `reason`, ktoré identifikuje príčinu. Najbežnejšie hodnoty:

- `ok` — prístup povolený.
- `unknown_reader` / `unknown_card` — čítačka alebo karta nie je registrovaná.

- `forbidden` — karta nemá rolu povolenú pre dané dvere.
- `replay` — detekované opakovanie sekvenčného čísla.
- `decrypt_failed` — AEAD overenie zlyhalo (poškodený frame, podvrhnutý nonce a pod.).
- `quarantined` — čítačka je v karanténe (zrušenie cez admin API).
- `bad_payload` / `bad_action` — frame má neplatný obsah.
- `remote_unavailable` — DecisionService nedostupný (po 3 retry pokusoch, fail-closed).
- `internal_error` — neočakávaná výnimka pri spracovaní (frame zamietnutý).

Chyby súvisiace s operačným systémom (`listen failed`, chyby SQLite, sieťové výnimky) sa vypisujú na štandardný výstup servera.

8 Príklad použitia

Typická prvá konfigurácia:

1. Spustiť server cez binárny súbor `access_admin` s konfiguračným YAML súborom ako argumentom.
2. Otvoriť webové rozhranie na adrese `localhost:8080`, zadať admin token (`admin`).
3. V sekcii *Readers* pridať čítačku s ID rovnakým ako `HW_READER_ID` v `hw_config.h`.
4. V sekcii *Door roles* pridať rolu (napr. `employee`) pre dvere s ID rovnakým ako `HW_DOOR_ID`.

5. V sekcii *Cards* pridať kartu — zadať UID v hex formáte (uppercase) a rolu **employee**.
6. Priložiť kartu k čítačke: pri správnej konfigurácii sa rozsvieti zelená LED a ozve sa krátke pípnutie.
7. Skontrolovať záznam v sekcii *Audit*.

Pre opätovné spustenie experimentov: `./build/experiments/s1_replay` (alebo iný scenár `s2–s7`); kompletný E2E beh cez `./experiments/run_e2e.sh`.